

САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ

Факультет прикладной математики – процессов управления

Кафедра информационных систем

Маслакова Анастасия Андреевна

Выпускная квалификационная работа

**«Веб-платформа интерактивной визуализации и
анализа многомерных численных алгоритмов»**

Уровень образования: бакалавриат

Направление: 01.03.02 Прикладная математика и информатика

Основная образовательная программа СВ.5005.2022: «Прикладная
математика, фундаментальная информатика и программирование»

Научный руководитель

доктор физ.-мат. наук, профессор

Матросов Александр Васильевич

Рецензент

Доцент кафедры моделирования экономических систем,

кандидат физ.-мат. наук

Ковшов Александр Михайлович

Санкт-Петербург

2026

Содержание

Введение	4
Постановка задачи	6
Обзор литературы	7
1 Технология WebAssembly и высокопроизводительные вычисления в браузерной среде	9
1.1 Определение и назначение WebAssembly	9
1.2 Архитектура и модель исполнения	10
1.3 Многопоточность в браузерной среде	11
1.4 Методы компиляции нативного кода в WebAssembly	11
1.5 Анализ производительности WebAssembly	15
1.6 Спецификации BLAS и LAPACK	15
2 Проектирование архитектуры платформы	17
2.1 Требования к платформе и функциональные возможности	17
2.2 Выбор технологического стека	18
2.3 Архитектура прототипа	19
2.4 Структуры данных и интерфейсы взаимодействия	22
3 Реализация прототипа	27
3.1 Подготовка вычислительного ядра	27
3.2 Подключение ядра к интерфейсному уровню	28
3.3 Модули-солверы	29
3.4 Точная арифметика рациональных дробей	31
3.5 Слой визуализации	32
3.6 Страницы-контейнеры	33
3.7 Графический модуль	34
4 Эксперимент и анализ производительности	36
4.1 Среда и методика измерений	36
4.2 Задержка инициализации платформы	37
4.3 Накладные расходы вызова ядра	38
4.4 Сравнение WebAssembly и JavaScript на одном алгоритме	38

4.5 Масштабирование времени решения с порядком задачи	40
4.6 Эффективность многопоточности	40
4.7 Стоимость точной арифметики	41
4.8 Издержки слоя визуализации	42
4.9 Обсуждение и сопоставление с известными результатами	43
Заключение	45
Список литературы	46

Введение

В современной вычислительной науке численные методы занимают центральное место. Методы линейной алгебры, разложение матриц и итерационные алгоритмы оптимизации составляют фундамент, на котором строятся решения в инженерии, физике, экономике, науке о данных и искусственном интеллекте. Глубокое понимание этих методов необходимо для специалиста, работающего с реальными вычислительными задачами. Именно численные алгоритмы лежат в основе широкого класса прикладных проблем: от решения систем линейных уравнений высокой размерности и вычисления собственных значений матриц до спектрального анализа многомерных сигналов и численного интегрирования дифференциальных уравнений.

Однако имеется существенный разрыв между потребностями специалистов и доступными инструментами. Традиционные среды для численного анализа, такие как: MATLAB, Scilab, Mathematica, обладают широкими возможностями, но их использование сопряжено с рядом ограничений. Приобретение коммерческих лицензий требует значительных финансовых затрат, локальная установка и настройка среды создают дополнительный барьер для входа, а сами инструменты, как правило, ориентированы на опытного пользователя и не предоставляют средств для пошаговой интерактивной визуализации вычислительного процесса. Это делает их малоприспособленными как для первичного освоения алгоритмов, так и для быстрого исследовательского прототипирования в условиях ограниченных ресурсов.

Существующие веб-ориентированные альтернативы, например онлайн - калькуляторы, вычислительные блокноты или образовательные платформы, лишь частично закрывают эту потребность. Вычислительные среды требуют знания языков программирования и настройки серверной инфраструктуры, онлайн - калькуляторы не обеспечивают работу с данными нетривиальной размерности, а большинство образовательных ресурсов ограничиваются статическими иллюстрациями и не позволяют пользователю взаимодействовать с алгоритмом напрямую.

Технология WebAssembly (Wasm) открывает принципиально новые возможности для заполнения этой ниши. Стандартизированный бинарный формат, поддерживаемый всеми современными браузерами, позволяет компилировать высокопроизводительные библиотеки численных вычислений в код, исполняемый непосредственно на стороне клиента с производительностью, сопоставимой с нативной. Это принципиально отличает WebAssembly от предшествующих попыток перенести вычисления в браузер. В сочетании с современными средствами веб-визуализации - WebGL, D3.js, Canvas API - данная технология создаёт техническую основу для реализации платформы нового класса.

Настоящая дипломная работа посвящена проектированию и разработке веб-платформы интерактивной визуализации и анализа многомерных численных методов. Платформа предоставляет пользователю единую браузерную среду, в которой становится возможным не только выполнять вычисления над многомерными данными с использованием проверенных численных библиотек, но и наблюдать за поведением алгоритмов в интерактивном режиме: визуализировать промежуточные состояния, исследовать зависимость результата от входных параметров, сравнивать методы по точности и вычислительной сложности. Подобный подход позволяет совместить доступность браузерного решения с вычислительной мощностью нативных библиотек и наглядностью, необходимой для глубокого понимания алгоритмической сути рассматриваемых методов.

Постановка задачи

Целью данной работы является создание прототипа платформы для интерактивного анализа и визуализации многомерных численных методов – полезного инструмента для образования, который сделает сложные алгоритмы наглядными и доступными для изучения.

Для достижения поставленной цели необходимо выполнить следующие задачи:

1. Провести анализ существующих инструментов и платформ для численного анализа и выработать на его основе требования к разрабатываемой платформе.
2. Исследовать технологии, применимые для реализации высокопроизводительных вычислений в браузере и обосновать выбор технического стека.
3. Спроектировать архитектуру и реализовать прототип платформы с поддержкой интерактивной визуализации выбранных численных методов и возможностью работы с многомерными входными данными.
4. Разработать методику оценки производительности платформы и провести вычислительный эксперимент в разных браузерах для сравнения эффективности созданной платформы.
5. Проанализировать полученные результаты и сформулировать рекомендации для дальнейшей доработки.

Обзор литературы

Формирование современных подходов к созданию высокопроизводительных вычислительных веб-приложений опирается на ряд исследований, посвящённых проектированию и стандартизации технологии WebAssembly, анализу её производительности, разработке линейно-алгебраических библиотек для веба, а также реализации интерактивной научной визуализации на стороне клиента. Анализ существующих публикаций позволяет обосновать выбор технологий и архитектурных решений, принятых в настоящей работе, и определить место разрабатываемой платформы среди существующих решений.

Фундаментальной работой, определившей развитие технологии WebAssembly, является статья [1], опубликованная на конференции PLDI 2017 и подготовленная инженерами из Google, Mozilla, Microsoft и Apple. В работе показано, что JavaScript, несмотря на оптимизации JIT-компиляторов, демонстрирует непредсказуемую производительность и не является надёжной целевой платформой для компиляции. WebAssembly решает эту проблему, предоставляя компактный бинарный формат с детерминированным поведением и скоростью выполнения, близкой к нативной.

Особый интерес представляют исследования, в который были проведены эксперименты по реализации матричных вычислений в WebAssembly и их сравнению с JavaScript-реализациями. Например, в отчете [2] реализовали библиотеку для работы с векторами и матрицами на Rust, скомпилированную в WASM через wasm-pack, и показали, что на операции умножения матриц размером до 1000×1000 WASM-реализация превосходит чистый JavaScript до 10 раз в Chrome и до 50 раз в Firefox. Кроме того, в работе [3] провели более детальный анализ на семи конфигурациях реализации умножения матриц (от 256×256 до 1024×1024), включая WebAssembly с SIMD-инструкциями и гибридные конфигурации с WebGPU, и выявили, что базовый WebAssembly без оптимизаций может быть даже медленнее JavaScript (ускорение 0.41x), поскольку JIT-компиляторы выполняют агрессивную оптимизацию паттернов доступа к памяти, - только комбинация SIMD-векторизации и блочного доступа к памяти позволяет WebAssembly превзойти JavaScript на 64%.

Ряд публикаций подтверждает техническую жизнеспособность компиляции существующих C/C++ библиотек в WebAssembly с использованием Emscripten для реализации интерактивной научной визуализации непосредственно в браузере. Так, в работе [4] скомпилировали libtiff и LAStools в WASM и продемонстрировали интерактивную визуализацию изоповерхности набора данных объёмом около 1 ТБ полностью в браузере с применением WebGPU: поверхность из 137,5 миллионов треугольников вычисляется за 526 мс и рендерится со скоростью 30 FPS. А в [5] описан подход к кроссплатформенной визуализации научных симуляций через WebAssembly и WebGL без внешних зависимостей,

где существующий C-код компилируется через Emscripten с поддержкой трёх режимов работы: нативного (OpenGL), гибридного (сервер + браузер) и полностью браузерного.

Опыт создания полноценных образовательных вычислительных сред в браузере представлен в статье [6]. Авторы реализовали браузерную среду разработки на Python, использующую Pyodide - порт CPython, скомпилированный в WebAssembly. Производительность системы составляет 3–5х от нативного CPython, что является приемлемым для образовательных задач. Практическое внедрение в школах Швейцарии подтверждает, что образовательные платформы на основе WebAssembly могут быть не только технически реализуемы, но и востребованы в учебном процессе.

Актуальность настоящего исследования подтверждается тем, что большая часть существующих работ либо фокусируется на отдельных аспектах, например, производительности или компиляции, либо ограничена специфическими кейсами без образовательной или визуализационной составляющей. Между тем, систематического сопоставления различных подходов в контексте создания комплексной вычислительной платформы с интерактивной визуализацией и образовательной направленностью в литературе крайне мало.

Рассмотренная литература подчёркивает как высокую значимость темы высокопроизводительных вычислений в браузере на основе WebAssembly, так и ограниченность имеющихся работ, объединяющих численные методы, интерактивную визуализацию и образовательную функциональность в комплексной, системной форме.

1 Технология WebAssembly и высокопроизводительные вычисления в браузерной среде

1.1 Определение и назначение WebAssembly

WebAssembly (Wasm) представляет собой открытый стандарт, определяющий переносимый бинарный формат инструкций для стековой виртуальной машины. Стандарт разработан рабочей группой W3C WebAssembly Working Group при участии инженеров компаний Google, Mozilla, Microsoft и Apple и опубликован в качестве рекомендации W3C в декабре 2019 года [1]. WebAssembly проектировался как целевая платформа компиляции для языков с явным управлением памятью, таких как C, C++ и Rust, и предназначен для выполнения вычислительно интенсивных задач в браузерной среде с производительностью, приближающейся к нативной.

Необходимость появления WebAssembly связана с эволюцией требований к веб-приложениям, которые всё чаще включают ресурсоёмкие вычисления: обработку графики, моделирование, анализ данных и выполнение алгоритмов, характерных для настольного программного обеспечения. Хотя современные JavaScript-движки, такие как V8, SpiderMonkey и JavaScriptCore, используют сложные механизмы оптимизации, включая JIT-компиляцию, специфика языка накладывает ограничения на достижимую эффективность и предсказуемость выполнения. В задачах, требующих строгого контроля над ресурсами и стабильной производительности, такие особенности могут становиться критическими.

WebAssembly предлагает альтернативную модель выполнения, ориентированную на эффективную доставку и исполнение кода в браузере. В отличие от JavaScript, распространяемого в текстовом виде и требующего многоступенчатой обработки, WebAssembly использует компактное бинарное представление, оптимизированное для быстрой загрузки и декодирования. Модуль WebAssembly компилируется в машинный код практически напрямую, минуя ряд промежуточных этапов, что сокращает время инициализации и снижает вариативность производительности во время выполнения. Такая архитектура делает WebAssembly особенно подходящим для реализации вычислительно интенсивных компонентов веб-приложений, приближая их характеристики к нативным.

1.2 Архитектура и модель исполнения

Архитектура WebAssembly базируется на концепции виртуальной стековой машины, что обеспечивает компактность инструкций и простоту их валидации. В отличие от традиционных регистровых архитектур, инструкции Wasm оперируют значениями, находящимися в стеке операндов, что позволяет эффективно транслировать код в машинные команды большинства современных CPU. Безопасность исполнения гарантируется строгой статической типизацией: корректность модуля проверяется на этапе декодирования, что полностью исключает возникновение ошибок несоответствия типов (type mismatch) во время выполнения.

Система типов WebAssembly ориентирована на максимальную производительность и включает четыре базовых числовых типа: целые числа (i32, i64) и числа с плавающей запятой (f32, f64). Для задач высокопроизводительных вычислений и линейной алгебры ключевое значение имеет поддержка типа f64, соответствующего стандарту IEEE 754. Это обеспечивает детерминированную семантику арифметических операций и гарантирует воспроизводимость численных результатов вне зависимости от браузера или аппаратной платформы хоста.

Одной из фундаментальных особенностей Wasm является модель линейной памяти. Она представляет собой непрерывный, расширяемый массив сырых байтов, полностью изолированный от управляемой памяти JavaScript. Такая модель напрямую соответствует адресному пространству языков C/C++ и Rust, что позволяет эффективно переносить низкоуровневые алгоритмы, активно использующие указатели и прямые манипуляции с памятью. Взаимодействие с внешней средой (JavaScript) осуществляется через буфер ArrayBuffer, предоставляющий быстрый доступ к данным без необходимости их сложной трансформации.

Структурно модуль WebAssembly организован в виде специализированных секций (типы, функции, глобальные переменные, экспорт/импорт). Такая организация поддерживает технологию потоковой компиляции (streaming compilation), позволяя браузеру начинать генерацию машинного кода еще до завершения загрузки бинарного файла по сети. Весь процесс выполнения протекает в изолированной программной среде («песочнице»). Прямой доступ к системным ресурсам или API браузера ограничен: модуль может взаимодействовать с внешним миром исключительно через явно импортированные функции, что обеспечивает высокий уровень безопасности при сохранении околонулевой скорости вычислений.

1.3 Многопоточность в браузерной среде

Браузерная среда исторически основана на однопоточной модели исполнения: основной поток обрабатывает пользовательские события, выполняет JavaScript-код и управляет отрисовкой интерфейса. Блокировка основного потока длительными вычислениями приводит к потере отзывчивости интерфейса. Механизм Web Workers позволяет создавать фоновые потоки исполнения, каждый из которых работает в изолированном контексте и обменивается данными с основным потоком через передачу сообщений.

Стандартный обмен данными между потоками основан на копировании, что создаёт существенные накладные расходы для больших числовых массивов. Объект `SharedArrayBuffer` решает эту проблему, предоставляя разделяемую область памяти, доступную из нескольких потоков одновременно. Синхронизация доступа обеспечивается набором атомарных операций через объект `Atomics`. В контексте WebAssembly эта архитектура позволяет транслировать вызовы стандарта POSIX `threads` непосредственно в Web Workers, размещая линейную память модуля в `SharedArrayBuffer` для параллельной обработки данных без избыточного копирования.

Использование разделяемой памяти в современных браузерах ограничено политикой кросс-доменной изоляции (`cross-origin isolation`). Техническая реализация доступа к объекту `SharedArrayBuffer` и высокоточным таймерам требует конфигурации среды окружения на уровне HTTP-протокола. Необходимым условием является установка заголовков `Cross-Origin-Embedder-Policy` (COEP) и `Cross-Origin-Opener-Policy` (COOP). Данная настройка формирует изолированный контекст безопасности, позволяющий выполнять многопоточные модули WebAssembly без риска несанкционированного доступа к данным из других контекстов.

1.4 Методы компиляции нативного кода в WebAssembly

WebAssembly как целевая платформа компиляции поддерживается широким набором инструментариев, различающихся по исходным языкам, архитектуре компилятора и характеристикам генерируемого кода. Выбор конкретного стека определяет архитектуру бинарного модуля, объём сопутствующего рантайма и методику взаимодействия с хост-средой. Рассмотрим основные методы компиляции.

АОТ-компиляция (`ahead-of-time`). Данный метод предполагает предварительную трансляцию исходного кода непосредственно в бинарный модуль WebAssembly ещё до запуска программы. Именно АОТ-компиляция стала ос-

новным способом доставки производительных Wasm-модулей в браузер и серверные рантаймы, поскольку она обеспечивает предсказуемое время инициализации, компактный формат поставки и высокий уровень оптимизации. К инструментам этого класса относятся Emscripten, LLVM/Clang, WASI SDK, Rust с `wasm-bindgen` и `wasm-pack`, TinyGo, AssemblyScript и другие компиляторные цепочки, генерирующие Wasm напрямую.

Плюсы	Минусы
Высокая производительность исполнения	Требуется отдельный этап сборки
Глубокие оптимизации на этапе компиляции	Плохо подходит для динамической генерации кода
Хороший контроль над экспортируемым интерфейсом	Может требовать сложной настройки памяти и рантайма
Удобная интеграция в стандартные сборочные пайплайны	Работа с внешними зависимостями бывает нетривиальной

Таблица 1.1: Преимущества и недостатки АОТ-компиляции

JIT-компиляция (just-in-time). При данном методе код компилируется во время исполнения, когда уже известен контекст запуска, профиль нагрузки и характеристики среды. В классическом виде JIT чаще применяется не как основной способ получения Wasm-модуля из исходного текста, а как механизм динамической компиляции промежуточного представления или байткода внутри среды выполнения. В экосистеме WebAssembly этот путь встречается заметно реже, чем АОТ, однако важен для рантаймов, где требуется адаптация к данным профилирования и возможность поздней оптимизации. Характерные инструменты и движки этого класса — Wasmer JIT, Cranelift в JIT-сценариях, а также внутренние JIT-пайплайны движков V8 и SpiderMonkey.

Плюсы	Минусы
Оптимизация с учётом реального профиля выполнения	Высокая сложность реализации
Возможность улучшать горячие участки после запуска	Увеличивает время холодного старта
Гибкая адаптация к среде исполнения	Требуется более сложный рантайм
Подходит для сценариев поздней оптимизации	Редко используется как основной метод доставки Wasm

Таблица 1.2: Преимущества и недостатки JIT-компиляции

Binary translation. Этот метод основан на переводе уже существующего бинарного кода, байткода или инструкций другой архитектуры в представление, пригодное для исполнения в среде WebAssembly. Такой подход применяется тогда, когда исходный код недоступен либо перенос системы с уровня исходников экономически нецелесообразен. В практических и исследовательских решениях сюда можно отнести CheerpX, некоторые QEMU/LLVM-ориентированные схемы динамической трансляции и другие специализированные системы двоичного переноса.

Интерпретация. В интерпретационном подходе в WebAssembly переносится не сама программа в виде нативно скомпилированного модуля, а ин-

Плюсы	Минусы
Позволяет использовать уже существующий бинарный код	Очень высокая сложность реализации
Не всегда требует доступа к исходникам	Производительность обычно ниже АОТ-компиляции
Полезен для переноса legacy-систем	Результат зависит от качества сопоставления архитектур
Может сокращать затраты на переписывание системы	Используется в основном в нишевых сценариях

Таблица 1.3: Преимущества и недостатки binary translation

терпретатор, который затем исполняет внешний бинарный код, байткод или сценарный язык инструкцией за инструкцией. Этот метод особенно удобен для обеспечения совместимости и быстрого запуска существующих экосистем без радикальной переработки их внутренней модели исполнения. К подобным решениям относятся Pyodide с интерпретатором CPython, Lua- и QuickJS-рантаймы, а также различные эмуляторы и интерпретаторы, скомпилированные в Wasm.

Плюсы	Минусы
Высокая гибкость	Производительность ниже АОТ-решений
Сравнительно быстрый перенос сложных экосистем	Часто уступает и JIT-подходам
Удобна для задач совместимости	Каждая инструкция проходит дополнительный уровень обработки
Позволяет сохранить полноту поддержки языка	Плохо подходит для тяжёлых вычислительных задач

Таблица 1.4: Преимущества и недостатки интерпретации

Hybrid-подход. Гибридная схема объединяет несколько методов компиляции и исполнения в рамках одной системы. На практике это означает, что часть кода может исполняться через интерпретацию, часть — быть заранее скомпилированной АОТ, а наиболее критические участки — дополнительно оптимизироваться специальным рантаймом. Подобные решения характерны прежде всего для управляемых платформ и больших фреймворков, где необходимо одновременно сохранить совместимость, приемлемую скорость запуска и достаточную производительность. Типичными примерами являются Blazor WebAssembly с АОТ-режимом, смешанные пайплайны Mono и ряд managed runtime-решений для Java и .NET.

Плюсы	Минусы
Гибкий баланс между скоростью запуска и производительностью	Архитектурно более сложен
Позволяет сочетать сильные стороны разных схем исполнения	Требует большего объёма рантайма
Удобен для крупных управляемых платформ	Сложнее в отладке и сопровождении
Помогает сохранять совместимость платформы	Итоговое поведение зависит сразу от нескольких механизмов

Таблица 1.5: Преимущества и недостатки hybrid-подхода

VM → Wasm. При данном методе в WebAssembly компилируется виртуальная машина или рантайм, а уже внутри него исполняется байткод исход-

ной платформы. В отличие от более общей интерпретации, здесь обычно речь идёт о полноценных управляемых экосистемах с собственным форматом байт-кода, системой типов, механизмами загрузки классов или сборки мусора. Этот путь особенно распространён для Java- и .NET-платформ, где требуется сохранить существующий стек библиотек и модель исполнения. Наиболее показательные примеры — Blazor WebAssembly с рантаймом Mono, Java-ориентированные решения вроде CheerpJ и другие системы, в которых виртуальная машина переносится в браузер как Wasm-модуль.

Плюсы	Минусы
Высокая совместимость с исходной экосистемой	Увеличивает размер поставляемого модуля
Можно повторно использовать существующий байткод	Даёт более медленный холодный старт
Сохраняет привычную модель исполнения платформы	Создаёт дополнительные накладные расходы рантайма
Облегчает перенос существующих библиотек	Эффективность зависит от качества реализации виртуальной машины

Таблица 1.6: Преимущества и недостатки метода $VM \rightarrow Wasm$

Сопоставление перечисленных методов показывает, что между ними существует устойчивый компромисс. Чем ближе метод к прямой предварительной компиляции, тем выше, как правило, производительность и тем меньше объём вспомогательного рантайма. Чем сильнее система опирается на интерпретацию, виртуальную машину или гибридную модель, тем проще сохранить совместимость с исходной платформой, но тем выше накладные расходы на запуск и исполнение. Сравнительные характеристики основных методов приведены в таблице 1.7.

Метод	Вход	Производительность	Сложность	Использование
АОТ	Исходный код	Высокая	Средняя	Основной способ сборки Wasm-модулей
JIT	Код во время выполнения, IR, байткод	Средняя	Высокая	Редко, в основном во внутренних рантаймах
Binary translation	Готовый бинарь, машинный код, байткод	Средняя или низкая	Очень высокая	Нишевые сценарии переноса legacy-систем
Интерпретация	Бинарь, байткод, сценарный код	Низкая	Низкая или средняя	Совместимость и быстрый перенос экосистем
Hybrid	Смешанный вход	Средняя	Высокая	Managed runtime и крупные платформы
$VM \rightarrow Wasm$	Байткод виртуальной машины	Средняя	Средняя	Java-, .NET- и другие байткодные экосистемы

Таблица 1.7: Сравнение методов компиляции в WebAssembly

1.5 Анализ производительности WebAssembly

Производительность WebAssembly зависит не только от самого формата, но и от характера программы, браузера, runtime-среды, компилятора и используемых оптимизаций. В работе [7], где сравнивались 41 C-бенчмарк, 9 пар WebAssembly/JavaScript-программ и 3 реальных приложения, показано, что WebAssembly не всегда автоматически превосходит JavaScript. Современные JIT-компиляторы эффективно оптимизируют JavaScript, тогда как прирост от JIT для WebAssembly в Chrome и Firefox оказался ограниченным. Кроме того, из-за модели линейной памяти WebAssembly в ряде случаев потреблял больше памяти, чем JavaScript [7].

В работе [8] дополнительно показано различие между микробенчмарками и реальными приложениями: на микробенчмарках WebAssembly в среднем был на 9.84% медленнее JavaScript и потреблял на 5.18% больше энергии, однако на реальных приложениях оказался на 17.24% быстрее и снизил энергопотребление на 24.34%. При сравнении с нативным кодом важным источником накладных расходов являются проверки границ линейной памяти: в работе [9] показано, что эффективные runtime-среды могут удерживать замедление WebAssembly относительно native на уровне 8–20%, но результат существенно зависит от реализации runtime.

Таким образом, WebAssembly следует рассматривать не как универсальный способ автоматического ускорения программ, а как эффективный инструмент для переноса вычислительно нагруженного и заранее оптимизированного кода в веб-среду или изолированную runtime-среду. Наибольший выигрыш достигается в случаях, когда исходный код уже хорошо оптимизирован на уровне алгоритмов и работы с памятью, а компиляция выполняется с учётом особенностей WebAssembly. Для задач линейной алгебры это особенно важно, поскольку итоговая производительность определяется не только самим фактом использования WebAssembly, но и блочной организацией вычислений, локальностью обращений к памяти, качеством компиляции и поддержкой низкоуровневых оптимизаций.

1.6 Спецификации BLAS и LAPACK

BLAS (Basic Linear Algebra Subprograms) представляет собой спецификацию интерфейсов для базовых операций линейной алгебры. Спецификация организована в три уровня:

- **Уровень 1** определяет операции вектор-вектор со сложностью $O(n)$: скалярное произведение, вычисление норм, масштабирование, линейные комбинации.

- **Уровень 2** определяет операции матрица-вектор со сложностью $O(n^2)$: умножение матрицы на вектор, решение треугольных систем.
- **Уровень 3** определяет операции матрица-матрица со сложностью $O(n^3)$: матричное умножение, ранговые обновления.

Операции третьего уровня обладают высоким соотношением вычислений к обращениям в память ($O(n)$ операций на элемент данных), что позволяет эффективно использовать кэш-память и достигать производительности, близкой к пиковой.

LAPACK (Linear Algebra PACKage) представляет собой библиотеку подпрограмм для решения задач линейной алгебры высокого уровня: факторизации матриц, решения систем линейных уравнений, вычисления собственных значений и сингулярного разложения. Подпрограммы LAPACK реализованы через вызовы BLAS, что позволяет автоматически наследовать оптимизации конкретной реализации BLAS для целевой платформы.

Соглашение об именовании подпрограмм LAPACK отражает тип данных и характер операции:

1. первая буква указывает на тип данных (например, **d** для double precision);
2. последующие буквы кодируют тип матрицы (**ge** для общей, **sy** для симметричной, **po** для положительно определённой);
3. окончание определяет выполняемую операцию (**sv** для решения системы, **trf** для факторизации).

OpenBLAS является одной из наиболее эффективных и широко используемых реализаций стандартов BLAS и LAPACK с открытым исходным кодом. Библиотека содержит глубоко оптимизированные вычислительные ядра, написанные на языках C, Fortran и ассемблере, что обеспечивает высокую производительность на различных процессорных архитектурах (x86, ARM, RISC-V и других). Благодаря поддержке многопоточного исполнения и возможности адаптации под специфические ограничения целевых сред, OpenBLAS выступает в качестве промышленного стандарта для интеграции методов высокопроизводительной линейной алгебры в современные программные комплексы и кроссплатформенные системы.

2 Проектирование архитектуры платформы

2.1 Требования к платформе и функциональные возможности

При проектировании интерактивной веб-платформы основной задачей становится построение архитектуры, сочетающей в себе высокую производительность и модульность с наглядной визуализацией алгоритмов. В отличие от обычных вычислительных программ, такая система ориентирована не только на получение результата, но и на демонстрацию логики вычислительного процесса непосредственно в браузере, что определяет её образовательное назначение - как инструмент изучения численных методов.

Исходя из этого назначения, к платформе предъявляются следующие функциональные требования:

- решение многомерных систем линейных алгебраических уравнений прямыми и итерационными методами;
- решение нелинейных систем уравнений итерационными методами;
- построение моделей аппроксимации и регрессии по набору наблюдений;
- гибкий ввод данных через матричные сетки, текстовые поля для уравнений и динамические таблицы наблюдений;
- пошаговая анимированная визуализация хода алгоритма с синхронной цветовой индикацией обрабатываемых элементов, управлением скоростью воспроизведения и выводом диагностических диаграмм.

К нефункциональным требованиям относятся: выполнение всех вычислений на стороне клиента, без серверного бэкенда; отсутствие блокировки основного потока браузера при обработке матриц прикладной размерности, что обеспечивает отзывчивость интерфейса; единый пользовательский сценарий взаимодействия для всех классов задач; возможность добавления нового численного метода без модификации существующих модулей.

Перечисленные требования определяют необходимость разделения платформы на несколько согласованных уровней. **Интерфейсный уровень** отвечает за ввод данных, навигацию и визуализацию результатов. **Прикладной уровень** управляет сценариями решения задач и координирует выпол-

нение конкретных численных методов. **Вычислительный уровень** обеспечивает выполнение ресурсоёмких операций линейной алгебры. Подобная декомпозиция минимизирует связанность подсистем, обеспечивая высокую масштабируемость и упрощая интеграцию новых расчетных модулей (как будет показано в 2.3) .

2.2 Выбор технологического стека

Проектирование платформы интерактивной визуализации численных алгоритмов требует обоснованного выбора технологий, обеспечивающих выполнение ключевых требований: высокопроизводительные вычисления на стороне клиента, модульная архитектура интерфейса, быстрая начальная загрузка и возможность интерактивной визуализации промежуточных состояний алгоритмов. В данном разделе последовательно рассматриваются решения, принятые для каждого уровня технологического стека.

Основу вычислительного ядра платформы составляет технология WebAssembly (WASM), выбор которой аргументирован результатами анализа в Главе 1. В качестве базового инструментария компиляции выбран Emscripten SDK, что продиктовано возможностью прямой трансляции высокооптимизированных нативных библиотек линейной алгебры, в частности OpenBLAS и LAPACK, в бинарный формат WASM. Данный подход позволяет использовать промышленно апробированные алгоритмы факторизации матриц и итерационных методов без потери их алгоритмической сложности и эффективности, достигая производительности, сопоставимой с нативными приложениями.

Для реализации пользовательского интерфейса и управления состоянием приложения выбран фреймворк React.js. Использование компонентно-ориентированного подхода и механизма Virtual DOM позволяет эффективно синхронизировать визуальное представление с результатами вычислений в реальном времени. Интеграция высокопроизводительного WASM-модуля в среду выполнения JavaScript осуществляется посредством асинхронного интерфейса загрузки, что предотвращает блокировку основного потока выполнения (Main Thread) и обеспечивает отзывчивость интерфейса при обработке массивов данных значительной размерности.

Визуализационный слой платформы базируется на использовании Canvas API. Данный выбор обоснован необходимостью отрисовки динамически изменяющихся многомерных структур, где традиционные методы манипуляции DOM-деревом демонстрируют избыточные вычислительные затраты. Взаимодействие JavaScript-слоя с вычислительным ядром на базе WebAssembly реализовано через управление линейной памятью модуля, что минимизирует накладные расходы на передачу данных и критически важно для обеспечения

интерактивности визуализации. При этом инфраструктура `SharedArrayBuffer` используется для обеспечения многопоточности в вычислительных операциях, формируя когерентную среду, способную решать задачи численного анализа в рамках веб-браузера с соблюдением требований к точности и быстродействию.

2.3 Архитектура прототипа

В предыдущих разделах были определены требования и технологический стек платформы. Настоящий раздел описывает архитектурные решения, принятые при проектировании: организацию взаимодействия между уровнями, механизм подключения алгоритмов, командный протокол вычислительного ядра и модель пошаговой визуализации.

Архитектура трёх уровней, обозначенных в разделе 2.1, следует паттерну MVP (Model–View–Presenter). Роль Model выполняет вычислительное ядро: оно инкапсулирует чистые вычисления и не зависит ни от интерфейса, ни от логики конкретного метода. Роль View принадлежит компонентам интерфейсного уровня: они отвечают за ввод данных и отображение результатов, но не содержат алгоритмической логики. Роль Presenter берут на себя модули-солверы прикладного уровня: каждый солвер координирует выполнение метода и управляет визуализацией, выступая единственным посредником между Model и View. Методы, ресурсоёмкие операции которых сводятся к стандартным процедурам линейной алгебры (прямые и итерационные методы для линейных СЛАУ, методы аппроксимации), делегируют их вычислительному ядру. Нелинейные методы (Ньютона, Бройдена, гомотопии, продолжения, простой итерации) реализуются на прикладном уровне без обращения к ядру для основного хода вычислений; для тех из них, что внутренне порождают линейные подсистемы, последние решаются собственными средствами прикладного уровня, а эквивалентная команда полного решения СЛАУ формируется лишь для отображения в журнале шагов в учебных целях. В соответствии с принципом MVP, компоненты интерфейса не обращаются к ядру самостоятельно: страница-контейнер получает абстрактный интерфейс вызова ядра из общего инфраструктурного контекста и передаёт его в контекст солвера, тогда как формирование команд и интерпретация результатов выполняются на стороне прикладного уровня.

Каждый численный метод оформлен как самодостаточный модуль фиксированной структуры — контракт солвера. Контракт включает общие метаданные метода, набор полей, специфичных для класса задачи, и операцию решения, реализующую полный вычислительный сценарий. Операция решения получает контекст выполнения — структуру, агрегирующую входные данные задачи, инжектируемый интерфейс вызова ядра, императивные интерфейсы визуализации и журнала шагов, разделяемый флаг пропуска анимации и флаг готовности



Рис. 2.1: Трёхуровневая архитектура платформы

ядра. Состав контракта приведён в таблице 2.1.

Страницы-контейнеры загружают модуль солвера динамически и передают его конфигурацию в стандартный набор компонентов. Добавление нового метода не требует модификации существующего кода: достаточно создать модуль с заданной структурой и зарегистрировать его в каталоге методов.

Взаимодействие прикладного уровня с вычислительным ядром организовано как текстовый командный протокол. Солвер формирует строку с именем BLAS- или LAPACK-подпрограммы и её параметрами, передаёт её через инжектируемый интерфейс и получает текстовый результат. Вся логика управления памятью ядра, кодирования команды и перехвата вывода инкапсулирована в адаптере и скрыта от солвера. Классы операций, используемые при реализации методов, приведены в таблице 2.2.

В отличие от подхода, при котором вычислительное ядро выполняет метод целиком и возвращает финальный результат, в данной архитектуре алгоритм реализован как последовательность элементарных операций на прикладном уровне. Каждая операция может включать один или несколько вызовов ядра, обновление визуального состояния и запись в журнал. Между операциями вставляются асинхронные паузы, которые передают управление браузеру и позволяют отрисовать промежуточное состояние до продолжения вычислений. Компоненты визуализации и журнала предоставляют солверу императивный интерфейс: методы для подсветки ячеек матрицы, поэлементного

Поле контракта	Обязат.	Назначение
<i>Общие поля (все классы задач)</i>		
Заголовок и описание метода	да	Отображаются на странице задачи
Уникальный идентификатор	да	Различение методов в общих компонентах визуализации
Базовая задержка между шагами журнала	да	Скорость интерактивной анимации регулируется отдельно
Дополнительные параметры метода	нет	ϵ , максимальное число итераций, релаксационный параметр и т.п.
Операция решения	да	Асинхронная процедура, получающая контекст выполнения
<i>Линейные СЛАУ и нелинейные системы (общие)</i>		
Размерности задачи и предзаполненного примера	да	
<i>Линейные СЛАУ</i>		
Предзаполненный пример	нет	Матрица коэффициентов и вектор правой части
Подпись поля ввода матрицы	нет	Используется при дополнительных требованиях к матрице
<i>Нелинейные системы</i>		
Предзаполненная система уравнений	нет	
Начальное приближение	нет	
<i>Аппроксимация и регрессия</i>		
Число входных признаков	нет	
Предзаполненный набор наблюдений	нет	
Минимально допустимое число точек	нет	По умолчанию определяется методом

Таблица 2.1: Контракт модуля-солвера

Класс операции	Назначение в контексте алгоритма
Поиск индекса экстремального элемента вектора	Выбор ведущего элемента
Линейная комбинация векторов	Элиминация при прямом ходе
Перестановка векторов	Перестановка строк при выборе ведущего элемента
Решение треугольной системы	Обратная подстановка
Полное решение СЛАУ	Независимая верификация результата
Решение симметрично-положительно-определённой СЛАУ	Метод Холецкого
Матрично-векторное произведение	Вычисление невязки и итерационных шагов
Скалярное произведение и норма	Контроль сходимости итерационных методов

Таблица 2.2: Классы операций вычислительного ядра, используемые в методах

обновления значений, добавления записей с описанием шага и соответствующей BLAS-командой. Пользователь может выбирать скорость воспроизведения или пропустить анимацию, при этом алгоритм доводит вычисления до конца и отрисовывает финальное состояние.

На интерфейсном уровне выделены три типа расчётных страниц-контейнеров, каждый из которых спроектирован под свой класс задач (помимо них в платформе присутствуют также вспомогательные навигационные страницы — главная и каталог методов). Страница линейных СЛАУ предоставляет матричную сетку $n \times n$ с вектором правой части и визуализирует процесс через интерактивную таблицу с подсветкой. Страница нелинейных систем использует текстовые поля для ввода уравнений и вектор начального приближения, отображая итерации в виде таблицы приближений. Страница регрессии предлагает динамическую таблицу наблюдений и визуализирует результат набором диаграмм. При этом все три типа страниц реализуют единый пользовательский сценарий: ввод и валидация на стороне формы → проверка готовности ядра → формирование контекста → вызов операции решения → отображение результата. Алгоритмическая логика полностью инкапсулирована в модуле-солвере.

Прямые методы решения линейных систем (Гаусса, LU, QR) поддерживают автоматический выбор между двумя режимами вычислений. Если все элементы входной матрицы и вектора правой части являются целыми числами, шаги алгоритма — выбор ведущего элемента, элиминация, обратная подстановка — выполняются в поле рациональных дробей на прикладном уровне, сохраняя точность на каждом промежуточном шаге. В противном случае используется арифметика с плавающей запятой, а перечисленные операции делегируются вычислительному ядру. В обоих режимах после получения решения выполняется независимая верификация через полное решение исходной системы средствами ядра. Точный режим особенно ценен в образовательном контексте: промежуточные результаты отображаются в виде обыкновенных дробей, что позволяет проверить каждый шаг вручную.

2.4 Структуры данных и интерфейсы взаимодействия

Данный раздел дополняет описание архитектуры со стороны организации данных. Основная задача проектирования в рамках этого контура заключается в определении форматов представления многомерных величин на каждом уровне и поиске механизмов их эффективной трансформации. Рассматриваются промежуточные структуры, необходимые для пошаговой визуализации, а также способы обеспечения согласованности данных между вычислительным ядром и графической подсистемой.

Логическое и линейное представление матриц

Между прикладным уровнем и вычислительным ядром данные пересекают архитектурную границу, на которой меняются их требования к памяти. На прикладном уровне естественной формой матрицы является массив строк, где каждая строка - отдельный массив чисел. Такое представление удобно для ручного ввода, поэлементного обновления, отрисовки в виде таблицы и отслеживания изменений конкретной ячейки в анимации. Спецификации BLAS и LAPACK, наоборот, предполагают непрерывное размещение элементов в одном блоке памяти, поскольку именно непрерывность обеспечивает эффективное использование кэш-памяти и блочных оптимизаций.

В архитектуре проектируется два согласованных представления матрицы:

- *Логическое (двумерное)* : существует на прикладном уровне и в журнале шагов. Используется для ввода, валидации, пошаговых преобразований и подсветки ячеек.
- *Линейное (одномерное)* : построчная (row-major) развёртка матрицы $n \times n$ в последовательность длины n^2 . Формируется только в момент вызова вычислительного ядра и используется как часть строки команды, передаваемой в линейную память WebAssembly.

Векторы во всех контекстах остаются одномерными и не требуют преобразования при передаче в ядро. Преобразование двумерного представления в линейное выполняется на стороне солвера непосредственно перед формированием команды, что позволяет хранить матрицы в удобной форме на всём остальном протяжении расчёта.

Типы рабочих данных

Помимо самих матриц и векторов, в системе проектируется ряд вспомогательных типов данных, специфичных для разных классов задач. Сводка этих типов приведена в таблице 2.3.

Тип данных	Назначение
Рациональное число	Пара числитель-знаменатель для режима точной арифметики; служит элементом матрицы и вектора при целочисленных входных данных.
Снимок шага	Запись текущего состояния алгоритма, формируемая между элементарными операциями.
Итерационная запись	Кортеж номера итерации, текущего приближения и значения нормы для отображения в журнале итерационных методов.
Набор наблюдений	Список строк (<i>признаки, значения</i>) для задач регрессии и аппроксимации.
Спецификация подсветки	Метаструктура с координатами выделяемых строк, столбцов и клеток, сопровождающая каждый снимок матрицы.

Таблица 2.3: Вспомогательные типы рабочих данных

Поскольку все числовые операции солвера выполняются через единый набор функций, переключение между рациональным и вещественным режимом происходит на уровне выбора реализации этих операций, а сам алгоритм работает с одной и той же структурой данных. Это устраняет дублирование логики между двумя режимами и позволяет точному режиму использовать тот же набор пошаговых снимков, что и режим с плавающей запятой.

Состав снимка шага

Реализация алгоритмов в виде последовательности атомарных операций (см. разд. 2.3) предполагает фиксацию промежуточных состояний системы. Таким состоянием является снимок шага — самодостаточный объект данных, передаваемый от вычислительного ядра к слою визуализации. В его состав входят:

- заголовок и описание операции в терминах линейной алгебры (например, элементарные преобразования строк или итерационное уточнение);
- текущее состояние объектов (матриц и векторов) в логическом представлении;
- метаданные подсветки: индексы ведущих, модифицируемых или целевых элементов (например, треугольных факторов);
- параметры сходимости (для итерационных методов): номер итерации, текущее приближение и норма невязки;
- строковая интерпретация команды ядра для отображения в техническом журнале.

Снимок проектируется как независимая структура, содержащая полный контекст для отрисовки. Это исключает обращение компонентов визуализации к глобальному состоянию алгоритма и позволяет синхронно обновлять различные представления — от анимированных таблиц на Canvas до текстового лога выполнения.

Структуры результатов по классам задач

Для обеспечения универсальности платформы структуры итоговых данных дифференцированы по типам решаемых математических задач. Каждая структура спроектирована таким образом, чтобы содержать исчерпывающую информацию для визуализации и верификации без необходимости повторного обращения к вычислительному ядру.

В задачах СЛАУ результат представляет собой агрегат, включающий вектор решения, факторы разложения (в случае использования прямых методов) и блок верификации. Наличие в структуре независимо вычисленного

эталонного решения и оценки невязки обеспечивает встроенный контроль корректности расчётов и позволяет пользователю мгновенно оценить точность полученного результата.

При работе с нелинейными системами результатом является массив итерационных состояний. Структура хранит последовательность пар «приближение - норма невязки», флаг сходимости, итоговое число выполненных итераций и детерминированную причину останова, такую как достижение заданной точности или исчерпание лимита итераций. Данная организация данных оптимизирована для автоматического формирования графиков сходимости и итерационных таблиц.

Для задач регрессии и аппроксимации используется композитная структура, разделенная на два функциональных блока. Аналитический блок содержит коэффициенты модели и статистические метрики качества, а графический блок включает предвычисленные значения функции на наборе узлов и данные для построения диаграмм рассеяния. Такое разделение позволяет одному объекту результата обслуживать несколько различных типов визуализаций, сохраняя при этом полный аналитический контекст расчёта.

Интерфейсы взаимодействия между уровнями

Спроектированные структуры данных функционируют в рамках четырех интерфейсов, фиксирующих границы между уровнями системы (табл. 2.4).

Интерфейс	Направление	Форма обмена	Режим
Страница – солвер	страница → солвер	объект контекста вызова	асинхронный, однократный
Контракт солвера	солвер → страница	метаданные и <code>solve(ctx)</code>	декларативный
Командный протокол ядра	солвер ↔ ядро	текстовая команда / текстовый ответ	синхронный, без состояния
Алгоритм – визуализация	солвер → слой отрисовки	журнал снимков и анимационные примитивы	поточковый, с прерыванием

Таблица 2.4: Интерфейсы взаимодействия между уровнями архитектуры

Интерфейс «страница-солвер» инкапсулирует параметры задачи в единый объект контекста. Это изолирует логику ввода от математических алгоритмов: страница управляет состоянием формы и дескрипторами Canvas, не участвуя в формировании команд ядра.

Контракт модуля-солвера унифицирует интеграцию методов через единый набор метаданных и исполняемую функцию `solve`. Декларативная часть контракта позволяет странице автоматически генерировать формы ввода, что упрощает масштабирование библиотеки без изменения ядра системы.

Командный протокол вычислительного ядра формализует вызовы WebAssembly-модуля. Солвер транслирует операции в текстовые команды (стандарты CBLAS/LAPACK), включая конфигурационные флаги и ли-

нейные массивы. Текстовый формат обеспечивает прозрачность логирования и упрощает отладку взаимодействия с низкоуровневой памятью.

Интерфейс «алгоритм-визуализация» связывает расчет с графическим слоем через два канала: журнал снимков и анимационные примитивы. Журнал гарантирует воспроизводимость состояний для экспорта, а примитивы обеспечивают динамическую подсветку элементов в процессе вычисления.

Предложенная схема интерфейсов делит платформу на слабо связанные слои. Это позволяет независимо модифицировать способы взаимодействия с пользователем, заменять вычислительные сборки ядра или внедрять новые типы графических отчетов без нарушения целостности системы.

3 Реализация прототипа

Настоящая глава посвящена программной реализации архитектурных решений, описанных в Главе 2. Основное внимание уделено сборке и подключению вычислительного ядра к интерфейсу, программной структуре солверов и реализации алгоритмов точной арифметики. Также приводится описание графической подсистемы и принципов взаимодействия между страницами-контейнерами и слоем визуализации..

3.1 Подготовка вычислительного ядра

Ядро реализовано как командная оболочка над OpenBLAS и LAPACK. Основной модуль `shell_cblas.c` определяет таблицу диспетчеризации `g_commands` и единную точку входа `run_command`, предназначенную для обработки текстовых команд. В соответствии с принятым протоколом (см. разд. 2.3), диспетчер выполняет токенизацию входной строки, идентификацию команды и передачу управления профильному обработчику `cmd_*`. В системе зарегистрировано 44 команды, охватывающие три уровня BLAS, ключевые подпрограммы LAPACK и операции для разреженных матриц (CSR). Распределение функций по категориям представлено в таблице 3.1.

Категория	N	Команды
BLAS Level 1	10	<code>ddot</code> , <code>dnrm2</code> , <code>dasum</code> , <code>idamax</code> , <code>dscal</code> , <code>daxpy</code> , <code>dcopy</code> , <code>dswap</code> , <code>drotg</code> , <code>drot</code>
BLAS Level 2	7	<code>dgemv</code> , <code>dtrsv</code> , <code>dger</code> , <code>dsymv</code> , <code>dsyr</code> , <code>dsyr2</code> , <code>dtrmv</code>
BLAS Level 3	5	<code>dgemm</code> , <code>dtrsm</code> , <code>dtrmm</code> , <code>dsyrk</code> , <code>dsymm</code>
LAPACK	17	<code>dgesv</code> , <code>dgetrf</code> , <code>dgetrs</code> , <code>dpotrf</code> , <code>dpotrs</code> , <code>dposv</code> , <code>dgeqrf</code> , <code>dorgqr</code> , <code>dormqr</code> , <code>dgels</code> , <code>dgelsd</code> , <code>dgelss</code> , <code>dgtsv</code> , <code>dptsv</code> , <code>dgbsv</code> , <code>dsyev</code> , <code>dgesvd</code>
Sparse (CSR)	3	<code>csr_spmv</code> , <code>csr_spmv</code> , <code>csr_to_dense</code>
Служебные	2	<code>threads</code> , <code>test</code>

Таблица 3.1: Категории команд вычислительного ядра

Для обеспечения отказоустойчивости ядра реализован механизм перехвата исключительных ситуаций. Парсинг параметров выполняется функциями `toint` и `todbl` с валидацией типов и диапазона. В случае возникновения критической ошибки внутри обработчика (некорректный аргумент, нехватка памяти, аномальный размер) управление передается через `longjmp` в точку возврата `g_cmd_abort`, установленную диспетчером. Такой подход предотвращает аварийное завершение WASM-модуля и позволяет вернуть в JavaScript-слой диагностическое сообщение. Результаты вычислений транслируются в стандартный вывод через `printf`, который перенаправляется окружением Emscripten в асинхронный поток данных приложения.

Процесс сборки автоматизирован скриптом `build.sh`. Библиотека

OpenBLAS компилируется под целевую платформу в режиме `TARGET=GENERIC` с поддержкой многопоточности (`USE_THREAD=1`, `NUM_THREADS=4`). Компоновка модуля `shell_cblas.c` с объектным файлом `libopenblas.a` выполняется компилятором `emcc` с оптимизацией `-O2`. Ключевые параметры компоновщика, определяющие конфигурацию среды исполнения, приведены в таблице 3.2. Итоговые артефакты включают бинарный модуль `shell_cblas.wasm`, связующий JavaScript-код и вспомогательный поток `shell_cblas.worker.js` для реализации `pthread`.

Параметр	Назначение
<code>-pthread, PTHREAD_POOL_SIZE=4</code>	Предсозданный пул из четырёх Web Workers под pthreads OpenBLAS
<code>INITIAL_MEMORY=67108864</code>	Начальный размер линейной памяти WASM (64 МБ)
<code>ALLOW_MEMORY_GROWTH=1</code>	Динамическое расширение памяти при необходимости
<code>INVOKE_RUN=0, EXIT_RUNTIME=0</code>	Ручной запуск <code>main</code> , рантайм не завершается после возврата
<code>EXPORTED_FUNCTIONS</code>	<code>_main, _run_command, _malloc, _free</code>
<code>EXPORTED_RUNTIME_METHODS</code>	<code>stringToUTF8, lengthBytesUTF8</code> (доступ из JS-обёртки)
<code>SUPPORT_LONGJMP=emscripten</code>	Поддержка <code>longjmp</code> для механизма обработки ошибок

Таблица 3.2: Ключевые параметры компоновщика `emcc`

3.2 Подключение ядра к интерфейсному уровню

Интеграция вычислительного модуля реализована в провайдере `WasmProvider` (`WasmContext.jsx`), который при монтировании инициализирует глобальный объект `window.Module`, задающий параметры рантайма Emscripten. В конфигурации задействованы три ключевых поля: функция `locateFile` обеспечивает корректную маршрутизацию запросов к бинарному файлу `shell_cblas.wasm` в целевой каталог проекта, поле `print` реализует прокси-функцию для перехвата и перенаправления стандартного вывода ядра, а обработчик `onRuntimeInitialized` сигнализирует о полной готовности среды исполнения путем выставления реактивного флага `wasmReady`. Такой подход позволяет централизованно управлять жизненным циклом WebAssembly-модуля и связывать низкоуровневый вывод C-кода с состоянием React-приложения.

Особенностью рантайма Emscripten является фиксация ссылки на метод `Module.print` в момент загрузки. Для обеспечения динамической смены потребителя данных реализован уровень косвенности: в замыкании контекста объявлена переменная-указатель `_printHandler`, а стабильный метод `Module.print` лишь транслирует аргументы в текущую активную функцию. Это позволяет перехватывать вывод ядра в любой момент жизненного цикла приложения простым переназначением `_printHandler`.

На данном механизме базируется функция `runBlas(cmd)`, реализующая текстовый протокол взаимодействия (см. разд. 2.3). С точки зрения прикладного уровня вызов `runBlas` является синхронным и атомарным, а вся низ-

коуровневая работа с памятью инкапсулирована внутри функции. Алгоритм её работы представлен в таблице 3.3.

Шаг	Действие
1	Сохранить текущий <code>_printHandler</code> и заменить его на накопитель строк
2	Вычислить длину UTF-8-представления команды функцией <code>lengthBytesUTF8</code>
3	Выделить буфер в линейной памяти WASM через <code>Module._malloc</code>
4	Записать строку в буфер функцией <code>stringToUTF8</code>
5	Вызвать экспортированную точку <code>Module._run_command</code> с указателем на буфер
6	Освободить буфер через <code>Module._free</code> , восстановить прежний <code>_printHandler</code> , вернуть накопленный текст

Таблица 3.3: Последовательность операций функции `runBlas`

Контекст экспортирует специализированный хук `useWasm`, предоставляющий компонентам реактивный доступ к состоянию `wasmReady` и функции `runBlas`. Это позволяет интерфейсным элементам вызывать методы линейной алгебры так же, как обычные функции JavaScript, полностью игнорируя специфику работы с WebAssembly.

3.3 Модули-солверы

Численные методы размещены в каталоге `src/hooks/solvers/`, разделённом на подкаталоги `linear/`, `nonlinear/` и `approx/` по классу решаемой задачи. Каждый модуль экспортирует объект, удовлетворяющий контракту солвера (табл. 2.1). Номенклатура реализованных модулей по классам приведена в таблицах 3.4–3.6. Значения столбца «Ядро»: *да* — ядро используется в основном ходе вычислений; *верификация* — ядро применяется только для контрольной проверки решения; *журнал* — ядро вызывается лишь для формирования эквивалентной BLAS-команды и независимой проверки.

Метод	Файл	Ядро	Точн. реж.
Метод Гаусса	<code>useGauss.js</code>	да	да
LU-разложение	<code>useLU.js</code>	да	да
QR-разложение	<code>useQR.js</code>	верификация	да
Метод Холецкого	<code>useCholesky.js</code>	да	нет
Метод Якоби	<code>useJacobi.js</code>	да	нет
Метод Зейделя	<code>useSeidel.js</code>	да	нет
SOR	<code>useSOR.js</code>	да	нет
MINRES	<code>useMinRes.js</code>	да	нет
BiCG	<code>useBiCG.js</code>	да	нет
GMRES	<code>useGMRES.js</code>	да	нет

Таблица 3.4: Модули-солверы для линейных СЛАУ (`linear/`)

Метод	Файл	Ядро	Точн. реж.
Метод Ньютона	useNewton.js	журнал	нет
Метод Бroyдена	useBroyden.js	журнал	нет
Простая итерация	useIteration.js	нет	нет
Гомотопия	useHomotopy.js	нет	нет
Продолжение по параметру	useContinuation.js	нет	нет

Таблица 3.5: Модули-солверы для нелинейных систем (nonlinear/)

Метод	Файл	Ядро	Точн. реж.
Линейная регрессия	useLinearRegression.js	да	нет
Полиномиальная регрессия	usePolyRegression.js	да	нет
Метод наим. квадратов	useLeastSquares.js	да	нет
RBF-интерполяция	useRBF.js	да	нет
Кубический сплайн	useSpline.js	да	нет

Таблица 3.6: Модули-солверы для аппроксимации и регрессии (approx/)

Маршруты регистрируются в `src/App.jsx`: словари `solverConfigs`, `nonlinearConfigs` и `approxConfigs` сопоставляют ключу маршрута динамический импорт соответствующего модуля. Страницы загружаются через `React.lazy` внутри `<Suspense>`, что обеспечивает разбиение бандла: модуль каждого солвера выделяется в отдельный `chunk` и подгружается только при переходе на маршрут.

Двухфазная схема (см. разд. 2.3) реализуется служебными точками. В интерактивной фазе между шагами вставляется ожидание `animSleep(viz.getSpeed(), skipRef)`; нажатие кнопки «Пропустить» устанавливает `skipRef.current = true`, после чего ветви алгоритма сворачиваются до окончательного состояния. В журнальной фазе те же шаги исполняются без задержек, а каждая операция фиксируется парой `stepLog.addStep(...)` и `await stepLog.showStep(...)`.

Композиция операций ядра по классам методов приведена в таблице 3.7.

Класс методов	Используемые операции ядра
Гаусс, LU	idamax, dswap, daxpy, dtrsv; верификация через dgesv
QR (Хаусхолдер)	только верификация решения через dgesv
Холецкий	dpotrf, dpotrs; верификация через dposv
Итерационные методы (Якоби, Зейдель, SOR, BiCG, MINRES, GMRES)	dgemv, daxpy, ddot; верификация через dgesv
Ньютон, Бroyден	внутреннее решение линеаризации через luSolve (JS); в журнал помещается эквивалентная команда dgesv для независимой проверки
Регрессия и аппроксимация	решение нормальных уравнений через dgesv

Таблица 3.7: Композиция операций ядра по классам методов

Прямые методы Гаусса и LU реализуют поэтапную элиминацию вызовами `idamax` (выбор ведущего элемента), `dswap` (перестановка строк), `daxpy` (обнуление поддиагональных элементов) и `dtrsv` (обратная подстановка). QR-разложение выполняется в JS методом отражений Хаусхолдера и обращается к ядру лишь для верификации; такой выбор обусловлен отсутствием в развёрнутой сборке OpenBLAS точек `dgeqrf/dorgqr`. Метод Холецкого опирается на пару LAPACK-вызовов `dpotrf/dpotrs` с проверкой через `dposv`. Итерационные методы выражают каждый внутренний шаг через `dgemv`, `daxpy` и `ddot`. Методы Ньютона и Бройдена решают линеаризацию внутренней процедурой `luSolve`, а в журнал шагов помещается эквивалентная команда `dgesv`, результат которой сравнивается с собственным для контроля. Регрессионные методы сводят задачу к нормальным уравнениям и решают их единственным вызовом `dgesv`. В `useGauss.js` дополнительно зарегистрирован обработчик `viz.enableBacksub(...)`, реализующий развёртку обратной подстановки в стиле Гаусса–Жордана; пользователь активирует её отдельной кнопкой.

Метод-специфичные параметры (точность ε , максимальное число итераций, релаксационный коэффициент ω в SOR, шаг τ в гомотопии и т. п.) передаются через поле `extraParams` контракта; их значения собираются компонентом `MatrixInput` и попадают в контекст солвера под именем `data.extra`.

3.4 Точная арифметика рациональных дробей

Точный режим реализован классом `Frac` в `src/utils/solverUtils.js`: пара целочисленных полей `n` (числитель) и `d` (знаменатель). Конструктор приводит знаменатель к положительному значению и сокращает дробь по наибольшему общему делителю, вычисляемому алгоритмом Евклида. Полный набор операций класса приведён в таблице 3.8.

Метод	Операция	Возвращаемый результат
<code>add(o), sub(o), mul(o), div(o)</code>	арифметика	новый объект <code>Frac</code>
<code>neg(), abs()</code>	унарные	новый объект <code>Frac</code>
<code>isZero()</code>	предикат	булево значение
<code>absVal()</code>	$ n/d $	число с плавающей запятой
<code>toNumber()</code>	n/d	число с плавающей запятой
<code>toString()</code>	сериализация	строка n/d , либо n при $d = 1$

Таблица 3.8: Операции класса `Frac`

Переключение режимов опирается на предикат `isAllInt(A, b)`: при целочисленных A и b входные данные конвертируются функциями `toFracMatrix` и `toFracVec`. Сам алгоритм солвера сохраняется единой функцией; различаются лишь определения арифметических операций, выбираемые тернарной развязкой по флагу `exact` (в типичной форме `const add = exact ? (a,b) => a.add(b) : (a,b) => a + b`). Такой приём устраняет дублирование между

режимами и позволяет точному режиму использовать ту же последовательность шагов визуализации, что и численному.

Для типографского отображения дробей в HTML предусмотрена функция `fmtNumHtml`: при $d \neq 1$ она формирует разметку `<span class="frac"><sup>n</sup>&frac1;<sub>d</sub></span>` с учётом знака. Поскольку ядро работает только с двойной точностью, верификация в режиме `Frac` выполняется по числовым значениям, полученным методом `toNumber`. Точный режим поддержан в трёх прямых методах решения СЛАУ: Гаусса, LU и QR.

3.5 Слой визуализации

Компоненты слоя визуализации построены как React-компоненты с императивным интерфейсом через `ref`. Такая схема выбрана для того, чтобы солвер мог выполнять длительный асинхронный сценарий, обращаясь к DOM напрямую, минуя React-рендер на каждом шаге. Через `forwardRef` и `useImperativeHandle` экспортируются методы, перечисленные в таблице 3.9.

Компонент	Метод	Назначение
Visualization	<code>show, hide</code>	показ и скрытие секции
	<code>getSpeed</code>	текущая скорость анимации (мс/шаг)
	<code>setStatus, setOpLabel</code>	заголовок и метка текущей операции
	<code>setContainerHTML, appendHTML</code>	полная или инкрементальная замена содержимого таблицы
	<code>querySelector, querySelectorAll</code>	выборка элементов в области таблицы
	<code>enableBacksub, disableBacksub</code>	регистрация и отключение второй секции (обратная подстановка)
	<code>setBacksubStatus, setBacksubContainerHTML, backsubQuerySelector</code> и др.	управление содержимым секции обратной подстановки
StepLog	<code>clear</code>	очистка журнала
	<code>addStep</code>	добавление скрытого шага с возвратом ссылки на DOM-элемент
	<code>showStep</code>	асинхронный показ шага через паузу <code>stepDelay</code> (Promise)
MatrixInput	<code>buildGrid</code>	построение сетки <code><input></code> -полей размером $n \times n$
	<code>readInput</code>	чтение данных с валидацией; результат <code>{ n, A, b, extra }</code> либо <code>null</code>
	<code>loadExample</code>	подстановка демонстрационных данных из контракта

Таблица 3.9: Императивные методы компонентов слоя визуализации

Скорость анимации задаётся пользователем в `Visualization` выпадающим списком со значениями 3200, 1600 и 800 мс на шаг (соответственно 0.25x, 0.5x, 1x) при значении по умолчанию 1600 мс. Атомарные операции над таблицей собраны в `solverUtils.js`: `renderImat` перерисовывает расширенную матрицу как HTML-таблицу с атрибутами `data-row/data-col`, `highlightCell` и `highlightRow` добавляют CSS-классы выделения, `updateCell` обновляет ячейку с учётом анимационной паузы, а `clearHighlights` снимает все выделения.

Все эти функции обращаются к DOM через `querySelector` объекта `viz`, что позволяет повторно использовать их и в основной таблице, и в таблице обратной подстановки.

В компоненте `StepLog` метод `addStep` создаёт скрытый `<div>` с заголовком, описанием, фрагментом таблицы и связанной BLAS-командой, а `showStep` ожидает `stepDelay` миллисекунд, снимает класс `step-hidden` и прокручивает журнал к новому шагу. Поскольку `showStep` возвращает `Promise`, темп журнала контролируется на стороне солвера через `await`.

Максимальный порядок задачи зависит от устройства. Тип устройства определяется хуком `useViewport` (пороги ширины окна 600 и 1024 пикселей), а соответствующие лимиты возвращают функции `maxMatrixSize`, `maxNonlinearSize` и `maxRegressionFeatures` (таблица 3.10).

Класс задачи	Mobile (≤ 600 px)	Tablet (≤ 1024 px)	Desktop (иначе)
Линейные СЛАУ (порядок)	5	8	11
Нелинейные системы (порядок)	4	7	10
Регрессия (число признаков)	3	4	5

Таблица 3.10: Адаптивные ограничения на размерность задач

Дополнительные средства отрисовки нелинейных задач (HTML-блоки якобиана, итерационные таблицы, прогрессбар по параметру гомотопии и т. п.) собраны в `src/utils/nlsRenderHelpers.js`.

3.6 Страницы-контейнеры

Каждая из трёх страниц-контейнеров реализует единый сценарий взаимодействия (см. разд. 2.3): чтение входных данных → валидация → проверка `wasmReady` → формирование контекста → `config.solve(ctx)` → отображение результата. Состав контекста, передаваемого солверу, приведён в таблице 3.11.

Поле	Источник	Назначение
<code>data</code>	результат <code>readInput</code>	входные данные задачи в логическом представлении
<code>runBlas</code>	хук <code>useWasm</code>	вызов вычислительного ядра
<code>viz</code>	<code>ref</code> компонента <code>Visualization</code>	императивный интерфейс визуализации
<code>stepLog</code>	<code>ref</code> компонента <code>StepLog</code>	журнал шагов
<code>skipRef</code>	локальный <code>useRef</code> страницы	разделяемый флаг пропуска анимации
<code>wasmReady</code>	хук <code>useWasm</code>	признак готовности ядра

Таблица 3.11: Состав контекста выполнения `ctx`

`SolverPage` обслуживает линейные методы. На монтировании страница загружает контракт через `configLoader`, монтирует `MatrixInput`, `Visualization` и `StepLog` с пропсами из контракта (`prefix`, `defaultSize`, `matrixLabel`, `extraParams`, `stepDelay`). Перед вызовом `config.solve(ctx)` страница читает данные методом `matrixRef.current.readInput()`, проверяет `wasmReady`,

очищает журнал, отключает секцию обратной подстановки и прокручивает контейнер в видимую область.

`NonlinearSolverPage` использует n текстовых полей для уравнений (синтаксис вида $x_1^2 + x_2^2 - 4$) и поле начального приближения x_0 . Парсинг и численное дифференцирование возложены на утилиты `src/utils/mathCore.js`: функция `buildFunction` компилирует строку выражения в JS-функцию через `Function` с предварительной валидацией символов по белому списку и подменой математических идентификаторов (`sin`, `cos`, `tan`, `exp`, `log`, `ln`, `sqrt`, `abs`, `pow`, `PI`, `E`) на их `Math`-аналоги; `evalF` вычисляет значение системы; `computeJacobian` строит якобиан методом центральных разностей с шагом $h = 10^{-8}$; `luSolve` решает СЛАУ с частичным выбором ведущего элемента для коррекции Δx .

`RegressionPage` формирует динамическую таблицу наблюдений со строками вида $(x_1, \dots, x_m, y)$, причём число признаков m выбирается в пределах таблицы 3.10. Минимальное число точек определяется методом `config.minPoints(m)`, специфичным для каждого солвера. После решения страница получает от солвера структуру `charts` с предвычисленными данными и в эффекте `useEffect` отрисовывает её вызовом `renderCharts(chartsRef.current, charts)`.

3.7 Графический модуль

Все диаграммы регрессионных методов отрисованы средствами стандартного 2D Canvas API без сторонних библиотек (`src/utils/chartRenderer.js`, 567 строк). Помощник `createCanvas(w, h, colors)` создаёт элемент с учётом `window.devicePixelRatio`: физический размер canvas увеличивается в `dpr` раз, после чего контекст немедленно масштабируется обратно вызовом `ctx.scale(dpr, dpr)`, что гарантирует чёткость линий и текста на дисплеях высокой плотности. Функция `getColors` читает атрибут `data-theme` корневого элемента документа и возвращает палитру для светлой или тёмной темы, благодаря чему диаграммы автоматически адаптируются при её смене.

Точка входа `renderCharts(container, charts)` получает контейнер и объект-описание набора графиков; для каждого присутствующего поля `charts.*` вызывается соответствующая специализированная функция отрисовки. Список поддерживаемых типов приведён в таблице 3.12.

Тип	Функция	Применение
Карточка с формулой модели	<code>renderEquationCard</code>	все методы регрессии
Таблица числовых метрик	<code>renderMetrics</code>	все методы регрессии
Рассеяние с моделью	<code>renderScatterChart</code>	одномерные регрессии ($m = 1$)
Частная зависимость	<code>renderPartialDependence</code>	многомерные регрессии ($m \geq 2$)
Предсказание против факта	<code>renderPredVsActual</code>	все методы регрессии
Остатки (рассеяние)	<code>renderResidualsScatter</code>	все методы регрессии
Гистограмма остатков	<code>renderResidualHistogram</code>	все методы регрессии
Коэффициенты модели	<code>renderCoefficientsChart</code>	линейная и полиномиальная регрессии
Тепловая карта корреляций	<code>renderCorrelationHeatmap</code>	многомерные регрессии

Таблица 3.12: Типы диаграмм графического модуля

Разделение модельной и графической частей результата (см. разд. 2.4) обеспечивается тем, что регрессионный солвер возвращает из `solve` единый объект, содержащий и параметры модели (коэффициенты, узловые значения, предсказания, остатки, метрики), и набор готовых к отрисовке структур. Благодаря этому одно вычисление обслуживает все диаграммы без повторного обращения к ядру, а смена состава визуализации сводится к изменению набора полей объекта `charts` и не затрагивает алгоритм.

4 Эксперимент и анализ производительности

Глава посвящена количественной оценке реализованной платформы. Эксперимент построен вокруг семи независимых измерений, каждое из которых отвечает на конкретный вопрос о пригодности WebAssembly как среды исполнения численных алгоритмов в браузере: задержка холодной инициализации (§4.2), накладные расходы единичного вызова ядра (§4.3), производительность WebAssembly относительно чистого JavaScript на эквивалентных алгоритмах (§4.4), масштабирование времени решения с порядком задачи (§4.5), эффективность многопоточного режима OpenBLAS (§4.6), стоимость точной рациональной арифметики (§4.7) и накладные расходы слоя визуализации (§4.8). Сводное обсуждение результатов и сопоставление с известными в литературе оценками вынесены в §4.9.

4.1 Среда и методика измерений

Эксперимент проводится на трёх классах устройств, типичных для целевого окружения платформы (таблица 4.1): персональный компьютер на macOS и два мобильных устройства — iOS и Android. Подобная конфигурация согласуется со схемой исследования энергопотребления WebAssembly в работе [5], в которой ключевые выводы получены сопоставлением десктопного и мобильного исполнения, а также с измерительной методикой [4], где сравнение JS и WebAssembly выполнено на широком спектре устройств.

Устройство	Платформа	Браузеры	Многопоточность
Ноутбук (Mac)	macOS, CPU/RAM/cores — <i>TODO</i>	Chrome, Firefox, Safari	да (4 потока)
Смартфон (Android)	Android <i>TODO</i> , ARM <i>TODO</i>	Chrome (мобильный)	да (если поддерживается)
Смартфон (iOS)	iOS <i>TODO</i> , Apple Silicon	Safari (мобильный)	ограниченная

Таблица 4.1: Состав экспериментального стенда

Инструментарий измерений. Хронометраж выполняется браузерным API `performance.now()` (разрешение — сотни наносекунд при включённой изоляции страницы). Этапы загрузки фиксируются через `PerformanceObserver` и `PerformanceNavigationTiming`, что позволяет разложить интервал от `navigationStart` до `onRuntimeInitialized` на стадии: загрузка HTML, парсинг JS, скачивание `shell_cblas.wasm`, компиляция модуля, инициализация рантайма, готовность пула `pthread`. Размер бинарника ядра фиксирован (466 КБ, см. §3.1) и одинаков для всех конфигураций.

Контроль повторяемости. Каждое измерение повторяется N раз ($N \geq 30$ для микробенчмарков и $N \geq 10$ для полных решений СЛАУ); сообщается медиана и межквартильный размах (IQR). Перед серией выполняется один прогон-разогрев, отбрасываемый при анализе. Для измерения холодной загрузки кэш браузера предварительно очищается (включая `Cache Storage` и память WebAssembly-модуля); для измерения операций после загрузки — наоборот, прогон выполняется на полностью прогретом V8/JavaScriptCore с целью изолировать издержки самого ядра. Дискретизация по типу устройства реализуется в верстке адаптивно (см. §3.5), но в эксперименте размерности фиксируются явно, а пороги по ширине окна не задействуются.

Ограничения. Все измерения проводятся над одной и той же сборкой `shell_cblas.wasm`, скомпилированной с параметрами из таблицы 3.2. Многопоточность OpenBLAS требует включения заголовков `Cross-Origin-Opener-Policy` и `Cross-Origin-Embedder-Policy` (для доступа к `SharedArrayBuffer`); их отсутствие на iOS-Safari в обычных условиях обработано фолбэком на однопоточный режим, что специально отмечено при анализе. Для всех экспериментов источник тестовых данных детерминирован: матрицы и векторы генерируются псевдослучайным генератором с фиксированным seed.

4.2 Задержка инициализации платформы

Эксперимент 1 фиксирует время от первой навигации к платформе до первого момента, когда становится возможным вызов `runBlas` (флаг `wasmReady`, см. §3.2). Эта величина характеризует пользовательский опыт и сопоставима с метрикой *ready time* ранее изученных браузерных IDE [11] и обобщённой оценкой запуска WebAssembly-приложений [10]. Согласно [1], типичное время компиляции WebAssembly-модуля порядка 466 КБ должно укладываться в десятки миллисекунд, однако совокупная задержка включает также сетевую загрузку, разбор JS-обёртки и подъём пула pthread.

В каждой пробе фиксируются пять отметок времени: T_{nav} (момент навигации), T_{html} (`DOMContentLoaded`), $T_{\text{wasm-fetch}}$ (завершение скачивания `shell_cblas.wasm`), T_{init} (вызов `onRuntimeInitialized`) и T_{ready} (выставление флага `wasmReady` в `WasmContext`). По разностям этих отметок эксперимент выделяет четыре стадии: *скачивание*, *компиляция*, *инициализация рантайма*, *подъём пула потоков*. Измерение проводится отдельно для холодного (свежий запуск, очищенный кэш) и тёплого (повторное открытие) сценариев. Результат представлен на рис. 4.1.

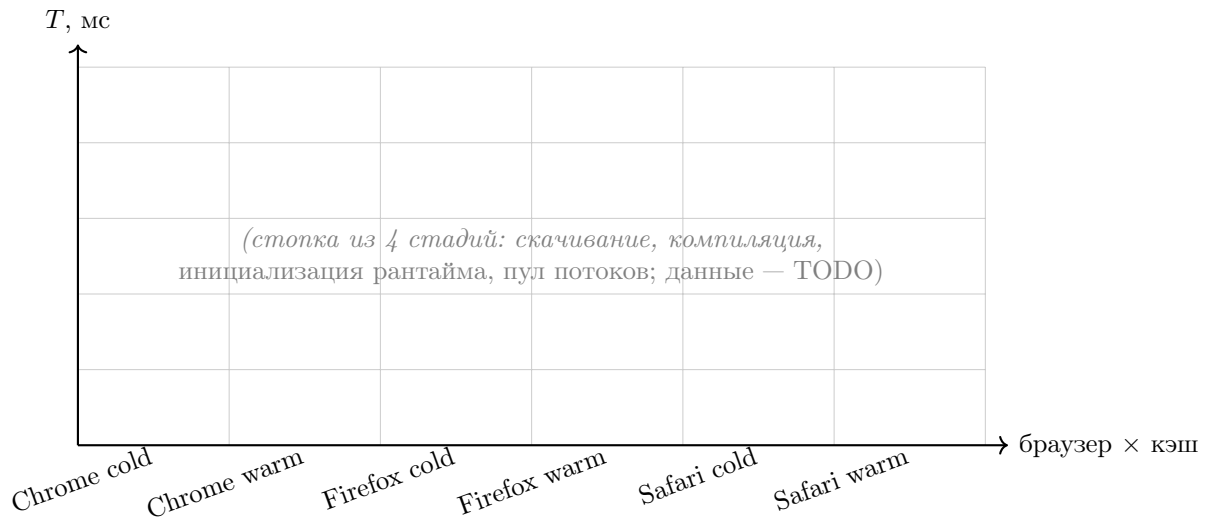


Рис. 4.1: Задержка инициализации платформы по стадиям, разбивка по браузерам и сценариям кэша

4.3 Накладные расходы вызова ядра

Эксперимент 2 измеряет фиксированную составляющую задержки одного вызова `runBlas` — последовательность из шести шагов (см. таблицу 3.3), исполняемая независимо от объёма передаваемых данных. Эта величина критична для алгоритмов, в которых вычисление складывается из большого числа мелких операций; в работе [3] показано, что именно граничные издержки `JS↔WASM` являются основным источником разрыва `WebAssembly` с нативной производительностью на коротких задачах, а в [7] подчёркнута важность их учёта при интеграции `WebAssembly` в производственные веб-приложения.

В качестве пробной операции выбрана `idamax` над вектором длины n , варьируемой по геометрической прогрессии ($n = 10, 50, 100, 500, 1000, 5000$). На каждом значении n выполняется $K = 1000$ повторов и сохраняется медианное время одного вызова. Для отделения собственно граничной составляющей от полезной работы дополнительно измеряется чисто JavaScript-эквивалент: проход по массиву поиском индекса максимального по модулю элемента. Разница между медианами при $n \rightarrow 0$ оценивает “пол” задержки — сумму расходов на `lengthBytesUTF8`, `_malloc`, `stringToUTF8`, `_run_command`, `_free` и перехват вывода. Результат — рис. 4.2.

4.4 Сравнение WebAssembly и JavaScript на одном алгоритме

Эксперимент 3 сопоставляет производительность ядра и чистого JavaScript на алгоритмически эквивалентной задаче — решении системы $Ax = b$ мето-

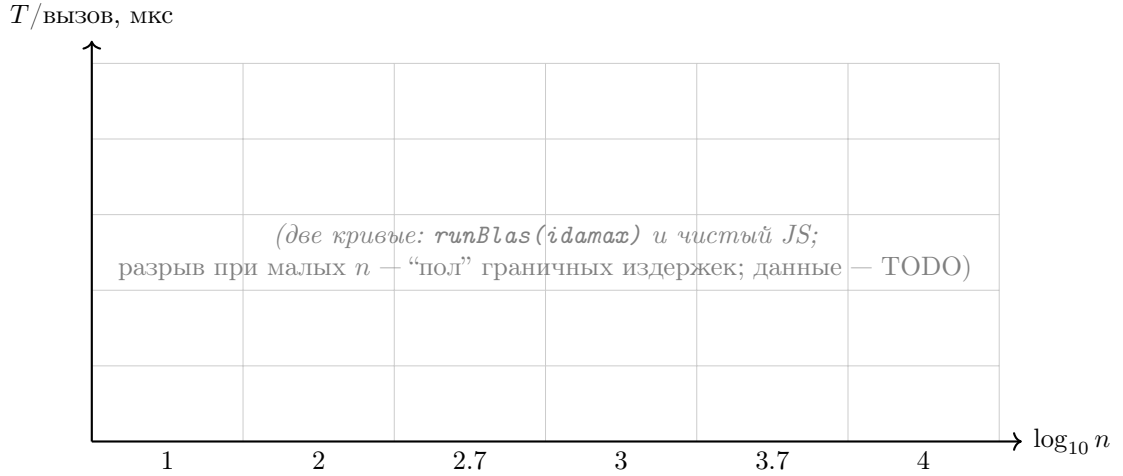


Рис. 4.2: Время одного вызова `runBlas(idamax)` в сравнении с эквивалентом на чистом JavaScript

дом исключения Гаусса с частичным выбором ведущего элемента. Аналогичные сравнения для матрично-векторных операций приведены в [4] и [6], причём оба источника отмечают существенное преимущество WebAssembly над оптимизированным JS на матричных задачах среднего и большого порядка; в [10] поставлен вопрос о том, насколько это преимущество устойчиво в реальных веб-приложениях.

WebAssembly-сторону представляет команда `dgesv` (полное решение СЛАУ через LAPACK). JavaScript-сторону — функция `luSolve` из `src/utils/mathCore.js` (LU-факторизация с частичным выбором ведущего, в коде проекта используется в нелинейных солверах — см. §3.6). Размерности: $n \in \{10, 50, 100, 200, 500, 1000\}$. На каждом n генерируется $M = 10$ случайных хорошо обусловленных матриц с фиксированным seed; время решения каждой матрицы измеряется отдельно. На рис. 4.3 приведено абсолютное время решения, на рис. 4.4 — ускорение T_{JS}/T_{WASM} как функция n .

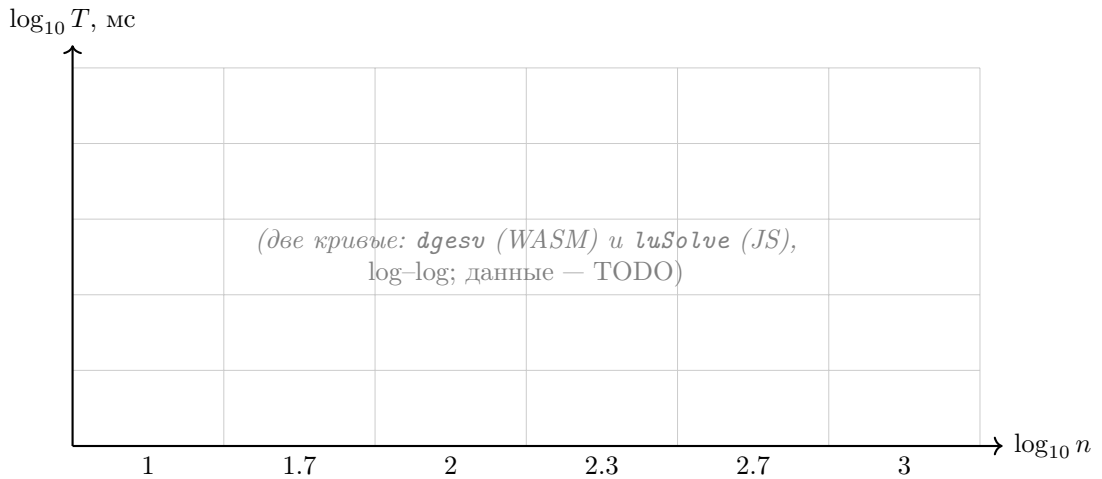


Рис. 4.3: Время решения СЛАУ при варьировании порядка матрицы

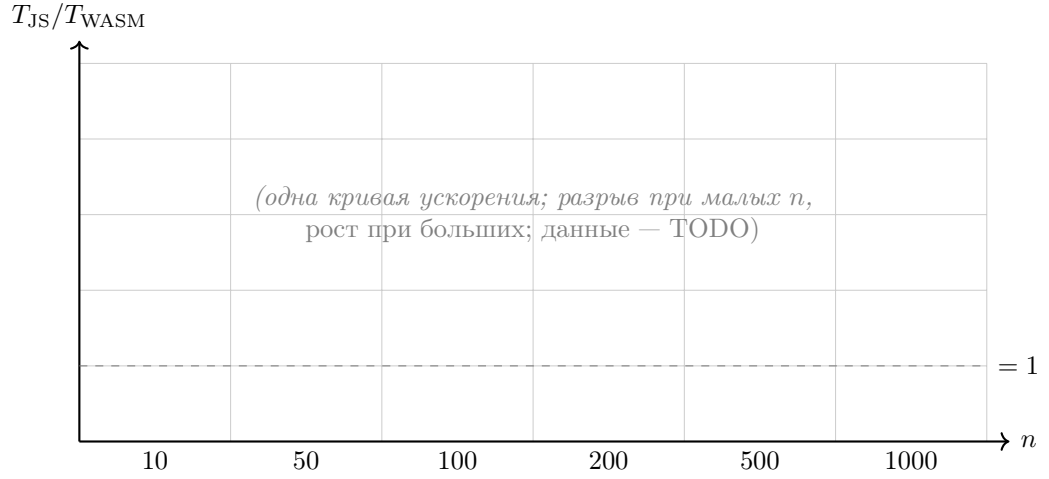


Рис. 4.4: Ускорение WebAssembly относительно JavaScript на эквивалентном алгоритме

4.5 Масштабирование времени решения с порядком задачи

Эксперимент 4 верифицирует асимптотическую сложность реализованных в платформе классов методов: прямого (Гаусс, $O(n^3)$) и итерационных (Якоби, GMRES, $O(n^2 \cdot k)$, где k — число итераций до сходимости). Цель — проверить, что наблюдаемый показатель степени α в законе $T(n) \propto n^\alpha$ согласуется с теоретической оценкой и что переход на WebAssembly не вносит дополнительных скрытых членов сложности; постановка близка по духу к [6]. Дополнительная мотивация — получение эмпирических ориентиров для пользователя при выборе метода в зависимости от размерности задачи.

Размерности $n \in \{10, 50, 100, 200, 500, 1000\}$ (Гаусс) и $n \in \{50, 100, 500, 1000\}$ (GMRES) при условии хорошей обусловленности. Время измеряется аналогично §4.4. По результатам каждого метода строится регрессия $\log T = \alpha \log n + \beta$, и оцениваемое значение α сравнивается с теоретическим. Результат — рис. 4.5.

4.6 Эффективность многопоточности

Эксперимент 5 оценивает выигрыш от параллельного исполнения OpenBLAS на ядре. Поддержка многопоточности заявлена через параметры сборки `USE_THREAD=1`, `NUM_THREADS=4`, `PTHREAD_POOL_SIZE=4` (см. таблицу 3.2); число активных потоков переключается командой ядра `threads n`. Постановка опирается на [1], где многопоточная модель WebAssembly обоснована как ключевой фактор приближения к нативной производительности на ресурсоёмких вычислениях, и на [5], где энергоэффективность обсуждается с учётом параллелизма.

В качестве ресурсоёмкой операции выбрана команда `dgemm` над квадратными матрицами порядка 500. Перебираются конфигурации $n_{th} \in \{1, 2, 4\}$



Рис. 4.5: Зависимость времени решения от порядка матрицы для прямого и итерационных методов

при прочих равных. На мобильных устройствах фиксируется фактическая поддержка многопоточности браузером (наличие `SharedArrayBuffer` и корректных заголовков `COOP/COEP`); при недоступности измерения проводятся в однопоточном режиме и эта особенность отмечается при анализе. Метрика — ускорение $S(n_{th}) = T(1)/T(n_{th})$. Результат — рис. 4.6.

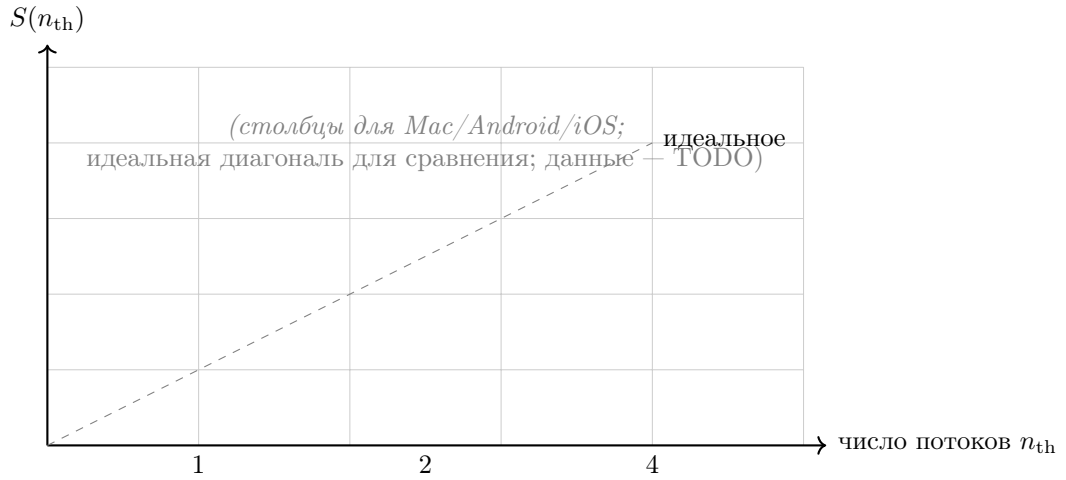


Рис. 4.6: Ускорение операции `dgemm` ($n = 500$) в зависимости от числа потоков OpenBLAS

4.7 Стоимость точной арифметики

Эксперимент 6 оценивает накладные расходы режима точной арифметики (см. §3.4). Поскольку задача поддержки рациональных дробей решается силами JavaScript, цена точности складывается из двух частей: стоимость обращений к классу `Frac` вместо нативных арифметических операций IEEE 754 и невозможность делегировать шаги алгоритма ядру. В литературе эта величина не

имеет прямого аналога, поскольку точная арифметика в JS-средах обычно реализуется отдельными библиотеками без интеграции с пошаговой визуализацией; настоящий опыт — оригинальный вклад работы.

Метод — Гаусс, режим выбирается предикатом `isAllInt` на этапе чтения данных. Размерности $n \in \{3, 5, 7, 9, 11\}$ (верхняя граница соответствует адаптивному лимиту настольной версии, таблица 3.10). На каждом n генерируется $M = 20$ случайных матриц с целыми элементами в диапазоне $[-9, 9]$ и проверяется выполнение точного равенства $Ax = b$ в поле $\mathbb{Q}$. Сравниваются медианы $T_{\text{exact}}(n)$ и $T_{\text{float}}(n)$, а также абсолютная ошибка решения относительно эталонного, полученного через `dgesv`. Результат — рис. 4.7.

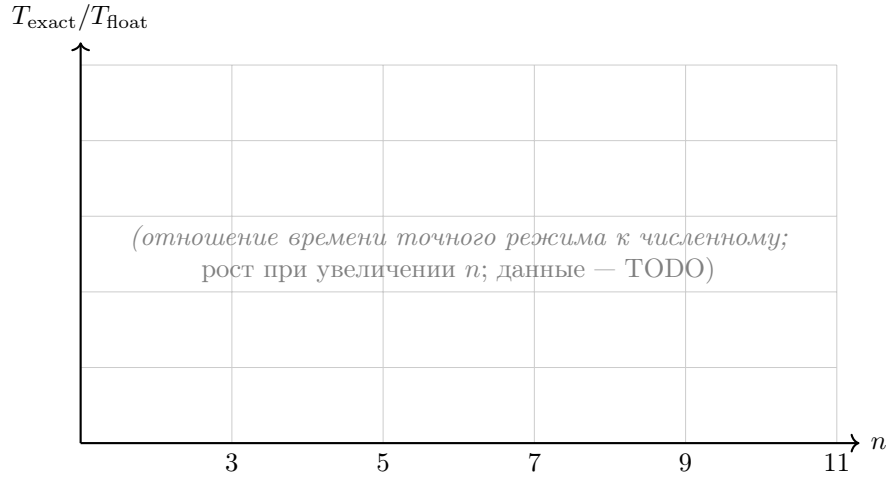


Рис. 4.7: Относительная стоимость режима точной арифметики (**Frac**) против численного режима

4.8 Издержки слоя визуализации

Эксперимент 7 разделяет полное время отклика метода на две составляющие: чистое вычисление и накладные расходы пошаговой визуализации (анимационные паузы, обновление DOM, журнал шагов). Постановка близка по идее к [8] и [9], где обсуждается стоимость интерактивной визуализации в браузере для научных приложений; ключевое отличие данной работы — зависимость общего времени от пользовательской настройки скорости анимации (3200/1600/800 мс на шаг, см. §3.5).

Сравниваются три сценария на одном и том же методе (Гаусс, $n = 8$, фиксированные данные): (а) *чистое вычисление* — выполнение алгоритма без обращений к `viz` и `stepLog`; (б) *визуализация с пропуском* — вызов `handleSkip` немедленно после старта (`skipRef.current = true`); (в) *визуализация на максимальной скорости* (800 мс на шаг). Время измеряется по `performance.now()` от начала `config.solve(ctx)` до завершения промиса. Результат — рис. 4.8.



Рис. 4.8: Разложение полного времени отклика метода на компоненты

4.9 Обсуждение и сопоставление с известными результатами

Полная интерпретация будет приведена после получения количественных данных. Ниже зафиксированы ожидаемые качественные выводы и точки сопоставления с литературой.

Загрузка и инициализация (§4.2). Согласно [1], характерное время компиляции WebAssembly-модуля размером порядка сотен килобайт составляет десятки миллисекунд; основной вклад в полную задержку ожидается со стороны сетевого скачивания и подъёма пула потоков. На тёплом кэше следует ожидать сокращения времени до десятков миллисекунд, что соответствует наблюдениям [11] для браузерных IDE.

Граничные издержки (§4.3). В работе [3] установлено, что фиксированный пол задержки одного вызова через границу JS↔WASM является доминирующим фактором замедления для коротких операций. Для текстового командного протокола настоящей работы пол ожидаемо выше, чем для прямого FFI, за счёт расходов на кодирование UTF-8 и аллокацию буфера; критическая размерность, при которой обращение к ядру становится оправданным, — предмет численной оценки в §4.3.

WebAssembly против JavaScript (§4.4–§4.5). Результаты [4] и [6] предсказывают существенное ($> 2\times$) преимущество WebAssembly при $n \gtrsim 100$. Настоящий эксперимент позволит локализовать пересечение, при котором WebAssembly начинает выигрывать у JavaScript-реализации с учётом граничных издержек, и оценить, насколько это значение согласуется с указанными работами на конкретной задаче решения СЛАУ.

Многопоточность (§4.6). Многопоточная модель WebAssembly [1] на персональных компьютерах ожидаемо даёт ускорение, близкое к идеальному при числе потоков, не превышающем число физических ядер. На мобильных

устройствах ожидается частичный регресс из-за тепловых ограничений и неполной поддержки `SharedArrayBuffer`, что согласуется с энергетическим анализом [5].

Точная арифметика (§4.7). Поскольку точный режим не использует ядро, увеличение времени должно следовать кубическому росту числа операций над `Frac` с дополнительным фактором роста знаменателей. Этот эксперимент — оригинальный вклад работы; полученные данные позволят уточнить рекомендуемые границы применимости точного режима в учебных сценариях.

Визуализация (§4.8). Доля анимационных пауз и DOM-обновлений в полном времени отклика характеризует пригодность платформы как интерактивного учебного инструмента; ожидается, что эта доля доминирует на малых n , где собственно вычисление занимает доли миллисекунды, и сокращается при увеличении размера задачи.

Окончательные выводы и количественное сопоставление с [1, 3, 4, 5, 6, 10] вынесены в Заключение после заполнения таблиц и графиков настоящей главы.

Заключение

Список литературы

- [1] Joseph Y-T. Leung Handbook of Scheduling: Algorithms, Models, and Performance Analysis CRC Press 2004
- [2] Michelle Galin, Anna Hedblom Linux Kernel Scheduler Evaluation for Performance-Critical Telecom Workloads Linköping University 2025
- [3] Jinsong Mao, Benjamin E. Ujcich, Shiqing Ma Rethinking Provenance Completeness with a Learning-Based Linux Scheduler arXiv 2025
- [4] Xinbo Wang, Shian Jia, Ziyang Huang, Jing Cao, Mingli Song Mixture-of-Schedulers: An Adaptive Scheduling Agent as a Learned Router for Expert Policies arXiv 2025
- [5] Ion Stoica, Hussein Abdel-Wahab Earliest Eligible Virtual Deadline First : A Flexible and Accurate Mechanism for Proportional Share Resource Allocation Old Dominion University 1996
- [6] A1349 Pavel Shago [Электронный ресурс]. – Режим доступа: <https://github.com/shgpavel/A1349> – Дата обращения 16.12.2025.
- [7] Andrew S. Tanenbaum, Herbert Bos MODERN OPERATING SYSTEMS Pearson Education 2023
- [8] Liz Rice Learning eBPF O'Reilly Media, Inc 2023
- [9] Torvalds L. Linux Kernel [Электронный ресурс] / L. Torvalds. – Режим доступа: <https://github.com/torvalds/linux> – Дата обращения 04.12.2025.
- [10] sched-ext/scx [Электронный ресурс]. – Режим доступа: <https://github.com/sched-ext/scx> – Дата обращения 07.12.2025.
- [11] lkml [Электронный ресурс]. – Режим доступа: <https://lkml.org> – Дата обращения 24.11.2025.
- [12] Jack Tigar Humphries, Neel Natu, Ashwin Chaugule, Ofir Weisse, Barret Rhoden, Josh Don, Luigi Rizzo, Oleg Rombakh, Paul Turner, Christoph Lameter ghOST: Scheduling Beyond the OS ACM SIGOPS 2021